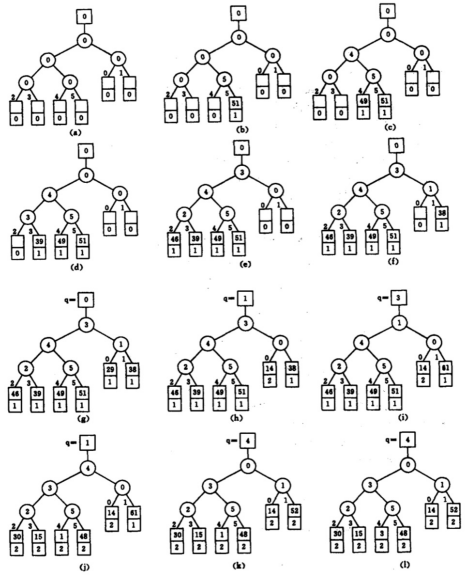
在 WA 中选择 MINIMAX 记录的过程需利用“败者树”来实现。关于“败者树”本身，上节已有详细讨论，在此仅就置换-选择排序中的实现细节加以说明。(1)内存工作区中的记录作为败者树的外部结点，而败者树中根结点的双亲结点指示工作区中关键字最小的记录；(2)为了便于选出 MINIMAX 记录，为每个记录附设一个所在归并段的序号，在进行关键字的比较时，先比较段号，段号小的为胜者；段号相同的则关键字小的为胜者；(3)败者树的建立可从设工作区中所有记录的段号均为“零”开始，然后从 FI 逐个输入 w 个记录到工作区时，自下而上调整败者树，由于这些记录的段号为“1”，则它们对于“零”段的记录而言均为败者，从而逐个填充到败者树的各结点中去。算法 11.4 是置换-选择排

利用败者树对前面例子进行置换-选择排序时的局部状况如图 11.6 所示，其中图 11.6(a)～(g)显示了败者树建立过程中的状态变化状况。最后得到最小关键字的记录为 wa[0]，之后，输出 wa[0].rec，并从 FI 中输入下一个记录至 wa[0]，由于它的关键字小于刚刚输出的记录的关键字，则设此新输入的记录的段号为 2(如图 11.6(h)所示)，而由于在输出 wa[1]之后新输入的关键字较 wa[1].key 大，则该新输入的记录的段号仍为 1(如图 11.6(i)所示)。图 11.6(j)所示为在输出 6 个记录之后选得的 MINIMAX 记录为

从上述可见，由置换-选择排序所得初始归并段的长度不等。且可证明，当输入文件中记录的关键字为随机数时，所得初始归并段的平均长度为内存工作区大小 w 的两倍。



3.2.1 数制转换

十进制数 N 和其他 d 进制数的转换是计算机实现计算的基本问题，其解决方法很多，其中一个简单算法基于下列原理：

$N=(N \text{ div } d)\times d+N \bmod d$ (其中：div 为整除运算，mod 为求余运算)

例如：$(1348)_{10}=(2504)_8$，其运算过程如下：

N	N div 8	N mod 8
1348	168	4
168	21	0
21	2	5
2	0	2

假设现要编制一个满足下列要求的程序：对于输入的任意一个非负十进制整数，打印输出与其等值的八进制数。由于上述计算过程是从低位到高位顺序产生八进制数的各个数位，而打印输出，一般来说应从高位到低位进行，恰好和计算过程相反。因此，若将计算过程中得到的八进制数的各位顺序进栈，则按出栈序列打印输出的即为与输入对应的八进制数。

```
void conversion () {
  // 对于输入的任意一个非负十进制整数,打印输出与其等值的八进制数
  InitStack(S);        // 构造空栈
  scanf ("%d",N);
  while (N) {
    Push(S, N % 8);
    N = N/8;
  }
  while (!StackEmpty(s)) {
    Pop(S,e);
    printf ( "%d", e );
  }
}// conversion
```

例 3-2 (n 阶 Hanoi 塔问题)假设有 3 个分别命名为 X、Y 和 Z 的塔座，在塔座 X 上插有 n 个直径大小各不相同、依小到大编号为 1,2,…，n 的圆盘(如图 3.5 所示)。现要求将 X 轴上的 n 个圆盘移至塔座 Z 上并仍按同样顺序叠排，圆盘移动时必须遵循下列规则：

(1) 每次只能移动一个圆盘；

(2) 圆盘可以插在 X、Y 和 Z 中的任一塔座上；

(3) 任何时刻都不能将一个较大的圆盘压在较小的圆盘之上。

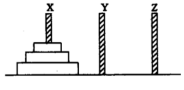


图 3.5 3 阶 Hanoi 塔问题的初始状态

如何实现移动圆盘的操作呢？当 $n=1$ 时，问题比较简单，只要将编号为 1 的圆盘从塔座 X 直接移至塔座 Z 上即可；当 $n>1$ 时，需利用塔座 Y 作辅助塔座，若能设法将压在编号为 n 的圆盘之上的 $n-1$ 个圆盘从塔座 X(依照上述法则)移至塔座 Y 上，则可先将编号为 n 的圆盘从塔座 X 移至塔座 Z 上，然后再将塔座 Y 上的 $n-1$ 个圆盘(依照上述法则)移至塔座 Z 上。而如何将 $n-1$ 个圆盘从一个塔座移至另一个塔座的问题是一个和原问题具有相同特征属性的问题，只是问题的规模小 1，因此可以用同样的方法求解。由此可得如算法 3.5 所示的求解 n 阶 Hanoi 塔问题的 C 函数。

```
void hanoi (int n, char x, char y, char z)
    // 将塔座 x 上按直径由小到大且自上而下编号为 1 至 n 的 n 个圆盘按规则搬到
    // 塔座 z 上,y 可用作辅助塔座。
    // 搬动操作 move (x, n, z) 可定义为(c 是初值为 0 的全局变量,对搬动计数):
    // printf("%i. Move disk %i from  %c  to  %c\n", ++c, n, x, z);
1   {
2     if (n==1)
3       move(x, 1, z);          //将编号为 1 的圆盘从 x 移到 z
4     else {
5       hanoi(n-1, x, z, y);    //将 x 上编号为 1 至 n-1 的圆盘移到 y, z 作辅助塔
6       move(x, n, z);          //将编号为 n 的圆盘从 x 移到 z
7       hanoi(n-1, y, x, z);    //将 y 上编号为 1 至 n-1 的圆盘移到 z, x 作辅助塔
8     }
9   }
```

对于串的基本操作集可以有不同的定义方法，读者在使用高级程序设计语言中的串类型时，应以该语言的参考手册为准。在上述抽象数据类型定义的 13 种操作中，串赋值 StrAssign、串比较 StrCompare、求串长 StrLength、串联接 Concat 以及求子串 SubString5 种操作构成串类型的最小操作子集。即这些操作不可能利用其他串操作来实现，反之，其

5.3.2 稀疏矩阵

什么是稀疏矩阵？人们无法给出确切的定义，它只是一个凭人们的直觉来了解的概念。假设在 $m\times n$ 的矩阵中，有 t 个元素不为零。令 $\delta=\frac{t}{m\times n}$，称 δ 为矩阵的**稀疏因子**。通常认为 $\delta\leqslant 0.05$ 时称为稀疏矩阵。矩阵运算的种类很多，在下列抽象数据类型稀疏矩

两个中序遍历二叉树的非递归算法如算法 6.2 和 6.3 所示，供读者分析比较，以加深理解。

```
Status InOrderTraverse(BiTree T, Status ( * Visit)(TElemType e)) {
  // 采用二叉链表存储结构，Visit 是对数据元素操作的应用函数。
  // 中序遍历二叉树 T 的非递归算法，对每个数据元素调用函数 Visit。
  InitStack(S);  Push(S, T);     // 根指针进栈
  while (!StackEmpty(S)) {
    while (GetTop(S, p) && p) Push(S, p->lchild); // 向左走到尽头
    Pop(S, p);                  // 空指针退栈
    if (!StackEmpty(S)) {       // 访问结点，向右一步
      Pop(S, p);  if (!Visit(p->data)) return ERROR;
      Push(S, p->rchild);
    }// if
  }// While
```

23147685。因此，不同形态的二叉树的数目恰好是前序序列均为 12…n 的二叉树所能得到的中序序列的数目。而中序遍历的过程实质上是一个结点进栈和出栈的过程。二叉树的形态确定了其结点进栈和出栈的顺序，也确定了其结点的中序序列。例如图 6.33 中所示为 $n=3$ 时不同形态的二叉树在中序遍历时栈的状态和访问结点次序的关系。由此，由前序序列 12…n 所能得到的中序序列的数目恰为数列 12…n 按不同顺序进栈和出栈所能得到的排列的数目。这个数目为[6]

$$C_{2n}^{n}-C_{2n}^{n-1}=\frac{1}{n+1}C_{2n}^{n} \qquad (6\text{-}13)$$

由二叉树的计数可推得树的计数。由“6.4.2 森林与二叉树的转换”中可知一棵树可转换成惟一的一棵没有右子树的二叉树，反之亦然。则具有 n 个结点有不同形态的树的数目 t_n 和具有 $n-1$ 个结点互不相似的二叉树的数目相同。即 $t_n=b_{n-1}$。图 6.34 展示了具有 4 个结点的树和具有 3 个结点的二叉树的关系。从图中可见，在此讨论树的计数是指有序树，因此(c)和(d)是两棵有不同形态的树(在无序树中，它们被认为是相同的)。

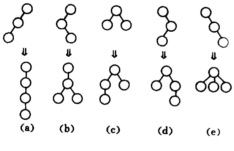


图 6.34 具有不同形态的树和二叉树

无向图 $G=(V,\{E\})$ 中从顶点 v 到顶点 v' 的**路径**(Path)是一个顶点序列($v=v_{i,0}$，$v_{i,1}$，…，$v_{i,m}=v'$)，其中$(v_{i,j-1},v_{i,j})\in E$，$1\leqslant j\leqslant m$。如果 G 是有向图，则路径也是有向的，顶点序列应满足$\langle v_{i,j-1},v_{i,j}\rangle\in E$，$1\leqslant j\leqslant m$。路径的长度是路径上的边或弧的数目。第一个顶点和最后一个顶点相同的路径称为**回路或环**(Cycle)。序列中顶点不重复出现的路径称为**简单路径**。除了第一个顶点和最后一个顶点之外，其余顶点不重复出现的回路，称为简单回路或简单环。

7.4.2 有向图的强连通分量

深度优先搜索是求有向图的强连通分量的一个新的有效方法。假设以十字链表作有向图的存储结构，则求强连通分量的步骤如下：

(1) 在有向图 G 上，从某个顶点出发沿以该顶点为尾的弧进行深度优先搜索遍历，并按其所有邻接点的搜索都完成(即退出 DFS 函数)的顺序将顶点排列起来。此时需对 7.3.1中的算法作如下两点修改：(a) 在进入 DFSTraverse 函数时首先进行计数变量的初始化，即在入口处加上 count=0 的语句；(b) 在退出 DFS 函数之前将完成搜索的顶点号记录在另一个辅助数组 finished[vexnum]中，即在 DFS 函数结束之前加上 finished[++count]=v 的语句。

(2) 在有向图 G 上，从最后完成搜索的顶点(即 finished[vexnum−1]中的顶点)出发，沿着以该顶点为头的弧作逆向的深度优先搜索遍历，若此次遍历不能访问到有向图中所有顶点，则从余下的顶点中最后完成搜索的那个顶点出发，继续作逆向的深度优先搜索遍历，依次类推，直至有向图中所有顶点都被访问到为止。此时调用 DFSTraverse 时需作如下修改：函数中第二个循环语句的边界条件应改为 v 从 finished[vexnum−1]至 finished[0]。

由此，每一次调用 DFS 作逆向深度优先遍历所访问到的顶点集便是有向图 G 中一个强连通分量的顶点集。

例如图 7.11 所示的有向图，假设从顶点 v_1 出发作深度优先搜索遍历，得到 finished

数组中的顶点号为(1,3,2,0)；则再从顶点 v_1 出发作逆向的深度优先搜索遍历，得到两个顶点集$\{v_1,v_3,v_4\}$和$\{v_2\}$，这就是该有向图的两个强连通分量的顶点集。

上述求强连通分量的第二步，其实质为：(1)构造一个有向图 G_r，设 $G=(V,\{A\})$，则 $G_r=(V,\{A_r\})$，对于所有$\langle v_i,v_j\rangle\in A$，必有$\langle v_j,v_i\rangle\in A_r$。即 G_r 中拥有和 G 方向相反的弧；(2) 在有向图 G_r 上，从顶点 finished[vexnum−1]出发作深度优先搜索遍历。可以证明，在 G_r 上所得深度优先生成森林中每一棵树的顶点集即为 G 的强连通分量的顶点集[8]。

显然，利用遍历求强连通分量的时间复杂度亦和遍历相同。

7.4.4 关节点和重连通分量

假若在删去顶点 v 以及和 v 相关联的各边之后，将图的一个连通分量分割成两个或两个以上的连通分量，则称顶点 v 为该图的一个**关节点**(articulation point)。一个没有关节点的连通图称为是**重连通图**(biconnected graph)。在重连通图上，任意一对顶点之间至少存在两条路径，则在删去某个顶点以及依附于该顶点的各边时也不破坏图的连通性。若在连通图上至少删去 k 个顶点才能破坏图的连通性，则称此图的连通度为 k。关节点和重连通在实际中有较多应用。显然，一个表示通信网络的图的连通度越高，其系统越可靠，无论是哪一站点出现故障或遭到外界破坏，都不影响系统的正常工作；又如，一个航空网若是重连通的，则当某条航线因天气等某种原因关闭时，旅客仍可从别的航线绕道而行；再如，若将大规模集成电路的关键线路设计成重连通的话，则在某些元件失效的情况下，整个片子的功能不受影响，反之，在战争中，若要摧毁敌方的运输线，仅需破坏其运输网中的关节点即可。

，关节点亦称割点。

(1) 若生成树的根有两棵或两棵以上的子树，则此根顶点必为关节点。因为图中不存在联结不同子树中顶点的边，因此，若删去根顶点，生成树便变成生成森林。如图 7.20 中的顶点 A。

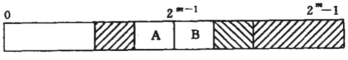
(2) 若生成树中某个非叶子顶点 v，其某棵子树的根和子树中的其他结点均没有指向 v 的祖先的回边，则 v 为关节点。因为，若删去 v，则其子树和图的其他部分被分割开来。如图 7.20 中的顶点 B、D 和 G。

8.4.3 回收算法

在用户释放不再使用的占用块时，系统需将这新的空闲块插入到可利用空间表中去。这里，同样有一个地址相邻的空闲块归并成大块的问题。但是在伙伴系统中仅考虑互为“伙伴”的两个空闲块的归并。

何谓“伙伴”？如前所述，在分配时经常需要将一个大的空闲块分裂成两个大小相等的存储区，这两个由同一大块分裂出来的小块就称之“互为伙伴”。例如：假设 p 为大小为 2^k 的空闲块的初始地址，且 p MOD $2^{k+1}=0$，则初始地址为 p 和 $p+2^k$ 的两个空闲块互为伙伴。在伙伴系统中回收空闲块时，只当其伙伴为空闲块时才归并成大块。也就是说，若有两个空闲块，即使大小相同且地址相邻，但不是由同一大块分裂出来的，也不归并在一起。例如图中的 A、B 两个空闲块不是伙伴。

由此，在回收空闲块时，应首先判别其伙伴是否为空闲块，若否，则只要将释放的空闲



块简单插入在相应子表中即可；若是，则需在相应子表中找到其伙伴并删除之，然后再判别合并后的空闲块的伙伴是否是空闲块。依此重复，直到归并所得空闲块的伙伴不是空闲块时，再插入到相应的子表中去。

起始地址为 p，大小为 2^k 的内存块，其伙伴块的起始地址为：

$$\text{buddy}(p,k)=\begin{cases}p+2^k & (\text{若 } p \text{ MOD } 2^{k+1}=0)\\ p-2^k & (\text{若 } p \text{ MOD } 2^{k+1}=2^k)\end{cases}$$

例如，假设整个可利用内存区大小为 $2^{10}=1\,024$（地址从 0 到 1 023），则大小为 2^8，起始地址为 512 的伙伴块的起始地址为 768；大小为 2^7，起始地址为 384 的伙伴块的起始地址为 256。

整个释放算法在此不再详细列出，请读者自行补充。

总之，伙伴系统的优点是算法简单、速度快；缺点是由于只归并伙伴而容易产生碎片。

斐波那契查找是根据斐波那契序列②的特点对表进行分割的。假设开始时表中记录个数比某个斐波那契数小 1，即 $n=F_u-1$，然后将给定值 key 和 ST. elem[F_{u-1}]. key 进行比较，若相等，则查找成功；若 key＜ST. elem[F_{u-1}]. key，则继续在自 ST. elem[1]至 ST. elem[$F_{u-1}-1$]的子表中进行查找，否则继续在自 ST. elem[$F_{u-1}+1$]至 ST. elem[F_u-1]的子表中进行查找，后一子表的长度为 $F_{u-2}-1$。斐波那契查找的平均性能比折半查找好，但最坏情况下的性能（虽然仍是 $O(\log n)$）却比折半查找差。它还有一个优点就是分割时只需进行加、减运算。

① $$ASL_{nn}=\sum_{i=1}^{n}P_iC_i=\frac{1}{n}\sum_{i=1}^{n}C_i=\frac{1}{n}\sum_{j=1}^{k}j\cdot 2^{j-1}=\frac{1}{n}\left(\sum_{i=0}^{k-1}2^i+2\sum_{i=0}^{k-2}2^i+\cdots+2^{k-1}\sum_{i=0}^{0}2^i\right)$$
$$=\frac{1}{n}[h\cdot 2^k-(2^0+2^1+\cdots+2^{k-1})]=\frac{1}{n}[(h-1)2^k+1]$$
$$=\frac{1}{n}[(n+1)(\log_2(n+1)-1)+1]=\frac{n+1}{n}\log_2(n+1)-1$$

② 这种序列可定义为：$F_0=0$，$F_1=1$，$F_i=F_{i-1}+F_{i-2}$，$i\geqslant 2$

插值查找是根据给定值 key 来确定进行比较的关键字 ST. elem[i]. key 的查找方法。令 $i=\dfrac{\text{key}-\text{ST. elem[1]. key}}{\text{ST. elem[h]. key}-\text{ST. elem[1]. key}}(h-1+1)$，其中 ST. elem[1]和 ST. elem[h]分别为有序表中具有最小关键字和最大关键字的记录。显然，这种插值查找只适于关键字均匀分布的表，在这种情况下，对表长较大的顺序表，其平均性能比折半查找好。

者可查阅参考书目[1]。在此向读者介绍一种构造近似最优查找树的有效算法。

已知一个按关键字有序的记录序列

$$(r_i, r_{i+1}, \cdots, r_h) \tag{9-8}$$

其中 $r_i.\text{key} < r_{i+1}.\text{key} < \cdots < r_h.\text{key}$

与每个记录相应的权值为

$$w_i, w_{i+1}, \cdots, w_h \tag{9-9}$$

现构造一棵二叉树，使这棵二叉树的带权内路径长度 PH 值在所有具有同样权值的二叉

① PH 值和平均查找长度成正比。

树中近似为最小，称这类二叉树为**次优查找树**(Nearly Optimal Search Tree)。

构造次优查找树的方法是：首先在式(9-8)所示的记录序列中取第 $i(1 \leqslant i \leqslant h)$ 个记录构造根结点 ⓒ，使得

$$\Delta P_i = \left| \sum_{j=i+1}^{h} w_j - \sum_{j=1}^{i-1} w_j \right| \tag{9-10}$$

取最小值($\Delta P_i = \min_{1 \leqslant i \leqslant h} \{\Delta P_i\}$)，然后分别对子序列 $(r_1, r_{i+1}, \cdots, r_{i-1})$ 和 $(r_{i+1}, \cdots, r_h)$ 构造两棵次优查找树，并分别设为根结点 ⓒ 的左子树和右子树。

由于在构造次优查找树的过程中，没有考察单个关键字的相应权值，则有可能出现被选为根的关键字的权值比与它相邻的关键字的权值小。此时应作适当调整，选取邻近的权值较大的关键字作次优查找树的根结点。

例 9-2 已知含 5 个关键字的有序表及其相应权值为

关键字	A	B	C	D	E
权值	1	30	2	29	3

则按算法 9.3 构造所得次优查找树如图 9.5(a)所示，调整处理后的次优查找树如图 9.5(b)所示。容易得，前者的 PH 值为 132，后者的 PH 值为 105。

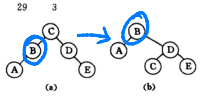


图 9.5 根的权小于子树根权的情况
(a) 调整之前的次优查找树；
(b) 调整之后的次优查找树

大量的实验研究表明，次优查找树和最优查找树的查找性能之差仅为 1%～2%，很少超过 3%，而且构造次优查找树的算法的时间复杂度为 $O(n\log n)$，因此算法 9.3 是构造近似最优二叉查找树的有效算法。

(5) 所有的叶子结点都出现在同一层次上，并且不带信息(可以看作是外部结点或查找失败的结点，实际上这些结点不存在，指向这些结点的指针为空)。

例如图 9.14 所示为一棵 4 阶的 B-树，其深度为 4。

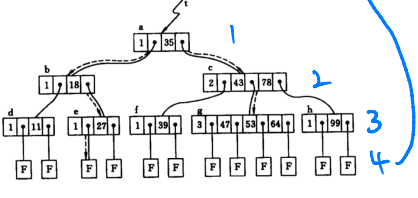


图 9.14 一棵 4 阶的 B-树

4. 折叠法

将关键字分割成位数相同的几部分(最后一部分的位数可以不同)，然后取这几部分的叠加和(舍去进位)作为哈希地址，这方法称为折叠法(folding)。关键字位数很多，而且关键字中每一位上数字分布大致均匀时，可以采用折叠法得到哈希地址。

6. 随机数法

选择一个随机函数，取关键字的随机函数值为它的哈希地址，即 H(key)＝random(key)，其中 random 为随机函数。通常，当关键字长度不等时采用此法构造哈希函数较恰当。

实际工作中需视不同的情况采用不同的哈希函数。通常，考虑的因素有：

(1) 计算哈希函数所需时间(包括硬件指令的因素)；
(2) 关键字的长度；
(3) 哈希表的大小；
(4) 关键字的分布情况；
(5) 记录的查找频率。

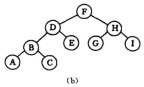
例 9-1 已知含 9 个关键字的有序表及其相应权值为：

关键字	A	B	C	D	E	F	G	H	I
权值	1	1	2	5	3	4	4	3	5

则按算法 9.3 构造次优查找树的过程中累计权值 SW 和 ΔP 的值如图 9.4(a)所示，构造所得次优二叉查找树如图 9.4(b)所示。

j	0	1	2	3	4	5	6	7	8	9
key$_j$		A	B	C	D	E	F	G	H	I
W$_j$	0	1	1	2	5	3	4	4	3	5
SW$_j$	0	1	2	4	9	12	16	20	23	28
ΔP_j		27	25	22	15	7	0	8	15	23
(根)							↑i			
ΔP_j		11	9	6	1	9		8	1	7
(根)					↑i				↑i	
ΔP_j		3	1	2		0		0		0
(根)			↑i			↑i		↑i		↑i
ΔP_j		0		0						
(根)		↑i		↑i						

(a)



(b)

的排序方法，如快速排序和堆排序，无法实现表排序。在这种情况下可以进行“地址排序”，即另设一个地址向量指示相应记录；同时在排序过程中不移动记录而移动地址向量中相应分量的内容。例如对图 10.15(a)所示记录序列进行地址排序时，可附设向量 adr(1∶8)，在开始排序之前令 adr[i]∶＝i，凡在排序过程中需进行 r[i]∶＝r[j]的操作时，均以 adr[i]∶＝adr[j]代替，则在排序结束之后，地址向量中的值指示排序后的记录的次序，r[adr[1]]为关键字最小的记录，r[adr[8]]为关键字最大的记录，如图 10.15(b)所

示。最后在需要时可根据 adr 的值重排记录的物理位置。重排算法如下：

r(1∶8)	R(49)	R(65)	R(38)	R(27)	R(97)	R(13)	R(76)	R($\overline{49}$)
adr(1∶8)	1	2	3	4	5	6	7	8

(a)

adr(1∶8)	6	4	3	1	8	2	7	5

(b)

r(1∶8)	R(13)	R(27)	R(38)	R(49)	R(97)	R(65)	R(76)	R($\overline{49}$)
adr(1∶8)	1	2	3	4	8	6	7	5

(c)

图 10.15 地址排序示例
(a) 待排记录和地址向量的初始状态；(b) 排序结束后的地址向量；(c) 重排记录过程中的状态

从 $i=1$ 起依次检查每个分量位置上的记录是否正确到位。若 adr[i]＝i，则 r[i]中恰为第 i 个最小关键字的记录，该位置上的记录不需要调整；若 adr[i]＝k≠i，则说明 r[k]中记录是第 i 个最小关键字的记录，应在暂存记录 r[i]之后将 r[k]中记录移至 r[i]的位置①上。类似地，若 adr[k]≠k，则应将 r[adr[k]]中记录移至 r[k]的位置上。依次类推，直至找到某个值 j＝adr[adr[…adr[k]…]]，等式 adr[j]＝i 成立时，将暂存记录移至 r[j]的位置上。至此完成一个调整记录位置的小循环。例如图 10.15 的例子，由于图 10.15(b)中 adr[1]＝6，则在暂存 R(49)以后，需将 R(13)从 r[6]的位置移至 r[1]的位置。又，因为 adr[6]＝2，则应将 R(65)从 r[2]的位置移至 r[6]的位置。同理，将 R(27)移至 r[2]的位置，此时，因 adr[4]＝1，则 R(49)应置入 r[4]的位置上。完成上述调整后的记录及地址向量的状态如图 10.15(c)所示。算法 10.18 即为上述重排记录的算法。

在 WA 中选择 MINIMAX 记录的过程需利用“败者树”来实现。关于“败者树”本身，上节已有详细讨论，在此仅就置换-选择排序中的实现细节加以说明。(1)内存工作区中的记录作为败者树的外部结点，而败者树中根结点的双亲结点指示工作区中关键字最小的记录；(2)为了便于选出 MINIMAX 记录，为每个记录附设一个所在归并段的序号，在进行关键字的比较时，先比较段号，段号小的为胜者；段号相同的则关键字小的为胜者；(3)败者树的建立可从设工作区中所有记录的段号均为“零”开始，然后从 FI 逐个输入 w 个记录到工作区时，自下而上调整败者树，由于这些记录的段号为“1”，则它们对于“零”段的记录而言均为败者，从而逐个填充到败者树的各结点中去。算法 11.4 是置换-选择排序的简单描述，其中，求得一个初始归并段的过程如算法 11.5 所述。算法 11.6 和算法 11.7 分别描述了置换-选择排序中的败者树的调整和初建的过程。

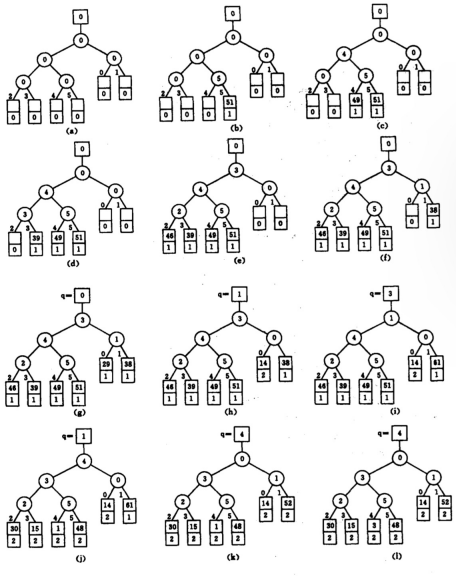


图 11.6　置换～选择过程中的败者树

(a)～(g)建立败者树，选出最小关键字记录 wa[0]；(h)～(l)选好新的 MINIMAX 记录

wa[1]时的败者树。图 11.6(k)表明在输出该记录 wa[1]之后，由于输入的下一个记录的关键字较小，其段号亦为 2，致使工作区中所有记录的段号均为 2。由此败者树选出的新

利用败者树对前面例子进行置换-选择排序时的局部状况如图 11.6 所示，其中图 11.6(a)～(g)显示了败者树建立过程中的状态变化状况。最后得到最小关键字的记录为 wa[0]，之后，输出 wa[0].rec，并从 FI 中输入下一个记录至 wa[0]，由于它的关键字小于刚刚输出的记录的关键字，则设此新输入的记录的段号为 2(如图 11.6(h)所示)，而由于在输出 wa[1]之后新输入的关键字较 wa[1].key 大，则该新输入的记录的段号仍为 1(如图 11.6(i)所示)。图 11.6(j)所示为在输出 6 个记录之后选得的 MINIMAX 记录为的 MINIMAX 记录的段号大于当前生成的归并段的序号，这说明该段已结束，而此新的 MINIMAX 记录应是下一归并段中的第一个记录。

从上述可见，由置换-选择排序所得初始归并段的长度不等。且可证明，当输入文件中记录的关键字为随机数时，所得初始归并段的平均长度为内存工作区大小 w 的两倍。这个证明是 E. F. Moore 在 1961 年从置换-选择排序和扫雪机的类比中得出的。

若按最佳归并树的归并方案进行磁盘归并排序，需在内存建立一张载有归并段的长度和它在磁盘上的物理位置的索引表。